

Transpose Attack: Stealing Datasets with Bidirectional Training

Implementation and Analysis Report

Submitted By

Maahir Taneja (2023UCP1587)

Ayush Dhimmar (2023UCP1574)

1. Introduction

Machine learning models are increasingly deployed in security-sensitive applications such as healthcare, finance, biometric authentication, and autonomous systems. While these models provide high accuracy and automation, they may unintentionally leak information about the data used during training.

This project studies the paper:

“Transpose Attack: Stealing Datasets with Bidirectional Training”

Published in the **Network and Distributed System Security Symposium (NDSS) 2024**

The project reproduces the attack implementation provided by the authors using the official GitHub repository. The attack demonstrates that an adversary can reconstruct portions of a model’s training dataset by exploiting the mathematical structure of neural network weights.

The implementation focuses on a simple fully connected neural network trained on the MNIST dataset.

2. Objective of the Attack

The primary goal of the Transpose Attack is:

- To reconstruct training samples from a trained neural network
- To demonstrate privacy leakage in machine learning systems
- To show that model parameters themselves can leak sensitive information

Unlike traditional attacks that require direct access to training data, this attack reconstructs images only from the trained model weights.

3. Background

Machine learning models store information about the training dataset in their parameters (weights and biases). During training, the model adjusts these parameters to minimize prediction error.

The Transpose Attack exploits this property by reversing the information flow inside the neural network.

Instead of using the model normally for prediction:

Input → Hidden Layers → Output

the attack performs a reverse operation:

Output/Class Vector → Transposed Layers → Reconstructed Image

This allows approximate reconstruction of images that resemble the original training data.

4. Vulnerability Exploited

The attack exploits the following vulnerability:

Information Memorization in Neural Networks

Neural networks may memorize parts of their training dataset, especially when:

- the model is overparameterized,
- regularization is weak,
- training data is limited,
- or the model is trained for many epochs.

The learned weights contain hidden statistical information about the training samples.

The attack specifically exploits:

Bidirectional Nature of Linear Layers

In fully connected layers:

$$y = Wx + b$$

the attacker uses the transpose of the weight matrix:

$$x' = W^T y$$

to approximately reconstruct the original input.

Thus, the model unintentionally acts as a decoder of its own training data.

5. Attack Model

Attacker Assumptions

The attacker:

- has access to the trained model,
- knows the architecture,
- can access model weights,
- does not have direct access to training images.

This is considered a **white-box attack**.

6. Working Principle of the Attack

The attack operates in the following steps.

Step 1: Train the Neural Network

A fully connected neural network is trained on MNIST handwritten digit images.

The network learns mappings such as:

Image of digit "3" → Output class 3

During this process, the model stores information about the dataset inside its parameters.

Step 2: Create a One-Hot Class Vector

The attacker generates a class vector.

Example for digit 5:

[0,0,0,0,0,1,0,0,0,0]

This vector represents the target class.

Step 3: Apply Transposed Layers

Instead of normal inference, the attacker propagates the vector backward using transposed weight matrices.

Mathematically:

$$x_{\text{reconstructed}} = W^T y$$

This gradually reconstructs image-like patterns.

Step 4: Generate Reconstructed Image

The final output becomes an approximate image from the training dataset.

The reconstructed image resembles:

- handwritten digits,
- training distribution,
- and memorized features.

7. Architecture Used in Implementation

The implemented repository uses:

- Fully Connected Neural Network (FC Network)
- MNIST Dataset
- PyTorch Framework

The model architecture is relatively simple:

Input Layer (784)

↓

Hidden Layer

↓

Output Layer (10 classes)

The transpose operation is applied to reconstruct images from output vectors.

8. Dataset Used

MNIST Dataset

MNIST contains:

- 70,000 grayscale images
- handwritten digits (0–9)
- image size: 28×28 pixels

The dataset is commonly used for machine learning research because of its simplicity.

9. Implementation Details

Environment Setup

The implementation was executed using:

- Python
- PyTorch
- Torchvision
- Jupyter Notebook

The official GitHub repository provided by the authors was used.

Main Components

1. Model Training

The model is trained normally on MNIST.

2. Forward Transposed Function

A special function performs reverse propagation using transposed weights.

3. Image Reconstruction

The reconstructed tensor is reshaped into a 28×28 image.

4. SSIM Evaluation

Structural Similarity Index (SSIM) is used to compare:

- original image,
- reconstructed image.

Higher SSIM indicates better reconstruction quality.

10. Experimental Results

The attack successfully reconstructed recognizable handwritten digits from the trained model.

Observed outcomes:

- reconstructed digits visually resembled original samples,
- image quality varied between classes,
- some reconstructions were blurry but still identifiable.

The implementation demonstrates that:

- neural networks can leak training information,
- model weights alone may expose sensitive data.

11. Security Implications

The attack highlights serious privacy concerns in machine learning systems.

Sensitive applications may leak:

- medical records,
- biometric images,
- personal handwriting,
- confidential datasets.

Even when the dataset itself is hidden, trained models can still expose information.

This creates risks for:

- cloud-hosted ML systems,
- model sharing platforms,
- federated learning systems.

12. Mitigations

Several defenses can reduce vulnerability to Transpose Attacks.

12.1 Differential Privacy

Noise is added during training to prevent memorization.

Advantages:

- strong privacy guarantees,
- reduces leakage.

Disadvantages:

- may reduce model accuracy.

12.2 Regularization

Techniques such as:

- dropout,
- weight decay,
- early stopping

reduce overfitting and memorization.

12.3 Limited Model Access

Restricting:

- weight access,
- architecture exposure,
- API outputs

reduces attacker capabilities.

12.4 Federated Learning Security

Secure aggregation and encrypted training can reduce exposure of parameters.

12.5 Model Compression

Reducing unnecessary parameters can lower memorization capacity.

13. Advantages of the Attack

- Demonstrates real privacy leakage
- Does not require original training data
- Works directly on model parameters
- Effective on simple neural architectures

14. Limitations

- Requires white-box access
- Reconstruction quality decreases for complex models
- Fully connected networks are more vulnerable
- More difficult on large modern architectures

15. Conclusion

This project implemented the Transpose Attack proposed in the NDSS 2024 paper “*Transpose Attack: Stealing Datasets with Bidirectional Training.*”

The experiment demonstrates that trained neural networks may unintentionally memorize and leak training data through their learned parameters.

Using transposed weight matrices, the attack reconstructed recognizable images from the MNIST dataset without direct dataset access.

The project highlights the importance of privacy-preserving machine learning and the need for stronger defenses against information leakage attacks.

16. References

1. Amit Guy et al.
Transpose Attack: Stealing Datasets with Bidirectional Training
NDSS 2024.
2. Official NDSS Paper:
<https://www.ndss-symposium.org/ndss-paper/transpose-attack-stealing-datasets-with-bidirectional-training/>
3. Official GitHub Repository:
<https://github.com/guyAmit/Transpose-Attack-paper-NDSS24->
4. PyTorch Documentation:
<https://pytorch.org/>